

AULA 2/4

JS: function

```
function add(x, y) {  
    var total = x + y;  
    return total;  
}
```

Se nada for explicitamente retornado, o valor de retorno é undefined.

Passagem de parâmetros:

```
> add()
```

```
NaN // You can't perform addition on undefined
```

```
> add(2, 3, 4)
```

```
5 // added the first two; 4 was ignored
```

JS: function

```
function add() {  
    var sum = 0;  
    for (var i = 0, j = arguments.length; i < j; i++) {  
        sum += arguments[i];  
    }  
    return sum;  
}  
  
> add(2, 3, 4, 5)  
14
```

JS: function expression (funções anônimas)

- Acontece quando uma função é criada sem ter um nome

```
var somaDoisNumeros = function (numero1, numero2) {  
    return numero1 + numero2;  
};  
somaDoisNumeros(10, 20);
```

JS: function expression (funções anônimas)

```
var avg = function () {  
    var sum = 0;  
    for (var i = 0, j = arguments.length; i < j; i+  
+) {  
        sum += arguments[i];  
    }  
    return sum / arguments.length;  
}
```

```
avg(5,10,5)  
6.66666...
```

JS: Template Literals

- Na hora de montar código JS é possível fazer uso do **Template Literals**

- Este mecanismo ajuda na escrita de código de forma mais fluida

```
let var = `I counted ${3+4} sheep`;
```

- Tudo que estiver dentro do `${ }` será interpretado como uma expressão JS.
- **ATENÇÃO**
 - É usado o character back-tick (```)
 - Se tentar utilizar aspa simples ou dupla, será lido como String.

JS: Spread

- Uso dos 3 pontinhos (...) in JS
- Algumas funções não aceitam que você passe arrays, sendo necessário passar os valores de forma individual
 - Por exemplo, a Math.max()
 - Dentro de um conjunto de valores, irá retornar o maior.
 - Math.max(1, 5, 77, 3, 55) □ Resultado: 77

```
const nums = [1,2,3,4,5]
Math.max(nums) // retorna NaN

Math.max(...nums) // retorna 5
// Mesmo que Math.max(1,2,3,4,5)
```

JS: Spread

- O operador spread (...) permite a iteração (como Array ou String) ser expandida em locais onde múltiplos elementos ou argumentos são esperados.

```
const array1 = [1, 2, 3];  
const array2 = [4, 5, 6];  
const mergedArray = [...array1, ...array2];  
// Retorna 1 Array:  
// [1, 2, 3, 4, 5, 6]
```

```
const mergedArray = [array1, array2];  
// Retorna 2 Arrays:  
// [ [ 1, 2, 3 ], [ 4, 5, 6 ] ]
```


JS: Spread

- Também permite concatenar elementos.

```
let cats = ['cat1', 'cat2'];  
let dogs = ['dog1', 'dog2']  
allPets = [...cats, ...dogs]  
// [ 'cat1', 'cat2', 'dog1', 'dog2' ]
```

- Spread também pode ser usado em objetos, copiando as propriedades de um objeto para outro.

```
felino = {pernas: 4, familia: 'Felidae'}  
novoObjeto = {...felino, cor: 'yellow'}  
// { pernas: 4, familia: 'Felidae', cor: 'yellow' }
```

JS: Spread

- Também é usado para juntar dois objetos
 - `let caninosFelinos = {...caninos, ...felinos}`
 - Soma os atributos dos dois objetos em um novo
- Quando há atributos com o mesmo nome, o valor do último parâmetro ganha, pois substitui o anterior.

```
felino = {pernas: 4, familia: 'Felidae'}  
canino = {dentes: 36, familia: 'Canidae'}
```

```
caninoFelino = {...felino, ...canino}
```

```
// { pernas: 4, familia: 'Canidae', dentes: 36 }
```

JS: arrow function

Há uma forma facilitada para declarar funções no JS... As **arrow functions**.

ANTES (MÉTODO TRADICIONAL)

```
var oldWay = function (name, nickname)
{
    return 'My name is ' + nickname +
    ', ' + name;
};

console.log(oldWay('James Bond',
'Bond'));
// My name is Bond, James Bond
```

DEPOIS (ARROW FUNCTION)

```
let newWay = (name, nickname) => {
    return 'My name is ' + nickname +
    ', ' + name;
};

console.log(newWay('James Bond',
'Bond'));
// My name is Bond, James Bond
```

CLASSES

JS: Classes Modernas

No JavaScript moderno, você pode criar **classes** usando a palavra-chave **class**, introduzida no ES6 (ECMAScript 2015).

Elas fornecem uma maneira mais clara e **estruturada** de criar objetos e trabalhar com **herança**, tornando o código mais legível.

EXEMPLO DE ESTRUTURA

```
class NomeDaClasse {  
  // Construtor: inicializa propriedades  
  constructor(param1, param2) {  
    this.propriedade1 = param1;  
    this.propriedade2 = param2;  
  }  
}
```

JS: Classes modernas

```
class NomeDaClasse {  
    // Construtor: inicializa propriedades  
    constructor(param1, param2) {  
        this.propriedade1 = param1;  
        this.propriedade2 = param2;  
    }  
    // Método da instância  
    metodoExemplo() {  
        console.log(`Propriedade1 é: ${this.propriedade1}`);  
    }  
    // Método estático (acessado diretamente pela classe, não pela instância)  
    static metodoEstatico() {  
        console.log("Este é um método estático.");  
    }  
}  
  
// Criando uma instância da classe  
const instancia = new NomeDaClasse("valor1", "valor2");  
instancia.metodoExemplo(); // Chamada do método da instância  
// Chamando um método estático  
NomeDaClasse.metodoEstatico();
```

JS: Classes modernas - Get e Set

```
class Produto {
  #preco; // privado de verdade

  constructor(nome, preco) {
    this.nome = nome;
    this.#preco = preco;
  }

  get preco() {
    return `R$ ${this.#preco.toFixed(2)}`;
  }

  set preco(novoPreco) {
    if (novoPreco > 0) {
      this.#preco = novoPreco;
    } else {
      console.log("O preço deve ser maior que zero.");
    }
  }
}

const produto = new Produto("Notebook", 2500);
console.log(produto.preco);
produto.preco = 3000;
console.log(produto.preco);
```

A depender da versão do JS, usa-se o #variavel para declarar uma variável privada.

Obs.: O Devtools possui uma permissão especial, se tentar acessar uma variável privada diretamente no Devtools você vai conseguir.

Teste em um arquivo HTML.

JS: Classes Modernas - Herança

```
class Animal {  
  constructor(nome) {  
    this.nome = nome;  
  }  
  falar() {  
    console.log(`${this.nome} faz um som.`);  
  }  
}
```

```
class Cachorro extends Animal {  
  falar() {  
    console.log(`${this.nome} late.`);  
  }  
}
```

// Instanciação e Uso

```
const dog = new Cachorro("Rex");  
dog.falar(); // "Rex late."
```

```
const animal = new Animal("Cobra");  
animal.falar(); // "Cobra faz um som."
```

Superclasse (Base)

A classe `Animal` define a estrutura base: um construtor para o nome e um método genérico `falar()`.

Subclasse (Herança)

A classe `Cachorro` estende `Animal` usando a palavra-chave `extends`.

Polimorfismo

A subclasse **sobrescreve** o método `falar()` para fornecer uma implementação específica.

JS: protótipo

Qualquer atributo ou função adicionado ao protótipo de uma dessas funções ficará disponível em qualquer objeto do tipo gerado por elas.

```
String.prototype.paraNumero = function () {  
    if (this == "um") {  
        return 1;  
    }  
}  
  
console.log("um".paraNumero()); // 1
```

JS: protótipo

```
var Pessoa = function (nome, email) {  
  this.nome = nome;  
  // verifica se o e-mail foi preenchido  
  if (email) {  
    this.email = email;  
  }  
}  
  
Pessoa.prototype.email = "contato@ufc.br"  
var ricardo = new Pessoa("Ricardo");  
console.log(ricardo.email); // contato@ufc.br  
  
var joao = new Pessoa("Joao da Silva",  
  "joao@da.silva");  
console.log(joao.email); // joao@da.silva
```

JS: protótipo

```
var Pessoa = function (nome, email) {  
    this.nome = nome;  
    // verifica se o e-mail foi preenchido  
    if (email) {  
        this.email = email;  
    }  
};  
  
Pessoa.prototype.fala = function () {  
    console.log("Olá, meu nome é " + this.nome + " e meu email é  
    " + this.email);  
};  
  
Pessoa.prototype.anda = function () {  
    console.log("Estou andando");  
};
```

EXERCÍCIO: Classes em JavaScript

Objetivo Geral

Crie uma estrutura de herança onde as classes **Gato**, **Cachorro** e **Pato** herdam da classe base **Animal**.

1. Classe Animal (Base)

- **Atributos:** nome e tutor.
- **Métodos:** EmitirSom() e Comer().
- **Encapsulamento:** Implemente métodos *Gets* e *Sets* para cada atributo.



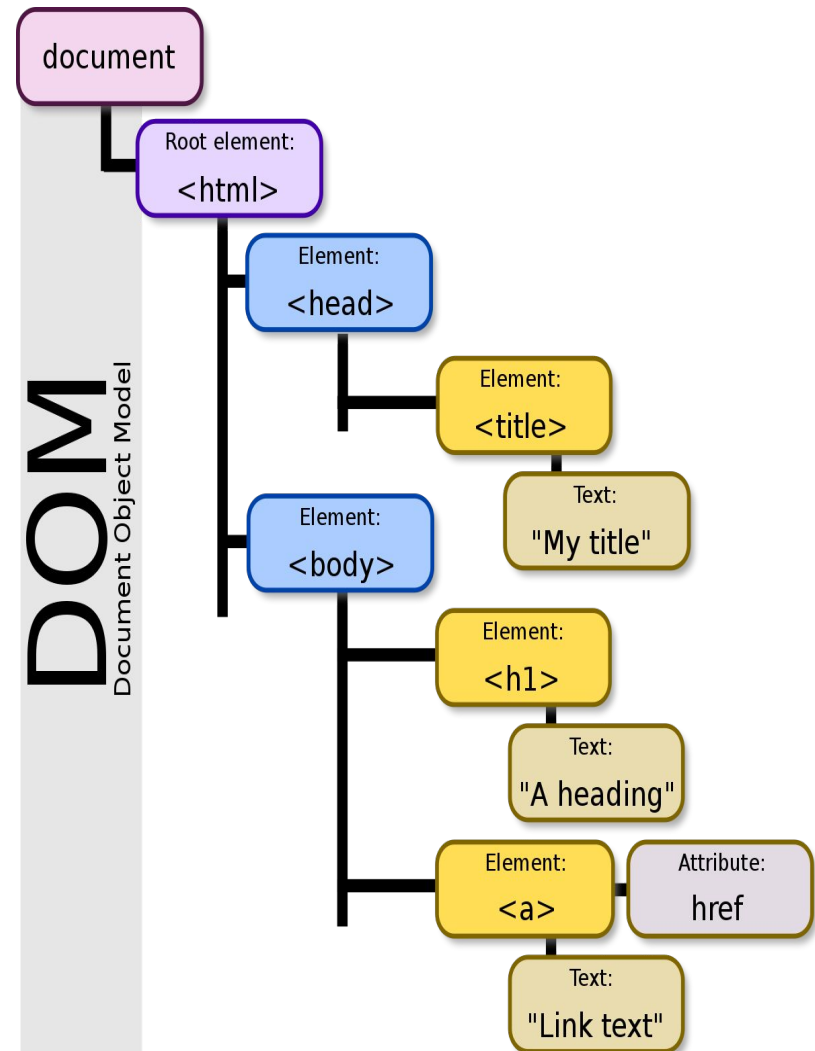
2. Implementação e Teste

- **Polimorfismo:** Cada animal deve emitir seu som específico: “Au au”, “Miau” ou “Quack”.
- **Instanciação:** Crie objetos de teste para validar o funcionamento das classes.

Document Object Model (DOM)

Document Object Model (DOM)

- O Document Object Model (DOM) é uma interface de programação que permite acessar os elementos HTML usando JavaScript. Com o acesso ao componente HTML é possível alterar estrutura, estilo e conteúdo.
- No DOM a página web é representada como nós e objetos.



DOM: Acessando Elementos

Métodos Tradicionais

Seleção por atributos específicos:

- `.getElementById()`
- `.getElementsByClassName()`
- `.getElementsByName()`
- `.getElementsByTagName()`

Seletores Modernos

Seleção flexível via seletores CSS:

- `.querySelector()`
- `.querySelectorAll()`

Estes métodos permitem buscar elementos usando a mesma sintaxe de seletores do CSS.

DOM: getElementById()

```
<p id="tituloPrincipal">Este é o título principal da minha  
página</p>
```

```
const elem = document.getElementById("tituloPrincipal");  
elem.style.color = 'red';  
elem.style.backgroundColor = 'blue'
```

ATENÇÃO !!!

Diferente do CSS, o JavaScript não permite hifens (-), por isso, alguns atributos possuem seu nome renomeado.

Por exemplo,

em JS usa-se **backgroundColor** ao invés de **background-color**

DOM: `getElementsByClassName()`

Permite selecionar todos os elementos que possuem uma ou mais classes.
Eles podem ser acessados como um array.

ESTRUTURA HTML

```
<span class="orange fruit">Orange  
Fruit</span>  
  
<span class="orange juice">Orange  
Juice</span>  
  
<span class="apple juice">Apple  
Juice</span>  
  
<span class="foo bar">Something  
Random</span>
```

JAVASCRIPT

```
// Somente seleciona os elementos  
que possuem as duas classes  
  
const allOrangeJuice =  
document.getElementsByClassName('orange  
juice');  
  
// Acessando o primeiro item do  
array resultante  
  
let result =  
document.getElementsByClassName('orange  
juice');  
result[0].style.color = 'red';
```

DOM: `getElementsByName()`

Retorna uma lista de elementos com o **atributo name** específico. Útil para capturar grupos de elementos relacionados, como botões de rádio.

```
<!-- HTML: Opções de um formulário -->
<input type="radio" name="genero" value="M" /> Masculino
<input type="radio" name="genero" value="F" /> Feminino

// JavaScript: Capturando todas as opções do grupo
const opcoes = document.getElementsByName("genero");
console.log(opcoes.length); // Resultado: 2
console.log(opcoes[0].type); // Resultado: "radio"

// Iterando sobre a NodeList retornada
opcoes.forEach(opt => console.log(opt.value));
```

DOM: `getElementsByTagName()`

Permite selecionar todos os elementos que pertencem a determinada **tag HTML**.

Eles podem ser acessados como um array.

```
const todosElementosTagP =  
document.getElementsByTagName("p");  
const tamanho = todosElementosTagP.length;  
  
console.log(tamanho);  
  
console.dir(todosElementosTagP);  
  
console.log(todosElementosTagP[0].innerText);
```

DOM: querySelector()

Permite selecionar o **primeiro** elemento que satisfaz a query, ou seja, o primeiro elemento que possui o seletor ou grupo de seletores especificados. Deve-se utilizar a sintaxe do CSS.

```
<h1>Exemplo de site</h1>
```

```
<p>Exemplo de tag p do HTML. </p>
```

```
<p class="elementop2">Exemplo de tag p do HTML 2. </p>
```

```
let minhaPrimeiraTagP = document.querySelector("p");  
console.log(minhaPrimeiraTagP);  
console.log(minhaPrimeiraTagP.innerText);
```

```
let minhaTagPByClass = document.querySelector(".elementop2");  
console.log(minhaTagPByClass);  
console.log(minhaTagPByClass.innerText);
```

DOM: querySelector()

- Principais propriedades do objeto retornado no querySelector.

PROPERTIES & METHODS

(the important ones)

- classList
- getAttribute()
- setAttribute()
- appendChild()
- append()
- prepend()
- removeChild()
- remove()
- createElement



- innerText
- textContent
- innerHTML
- value
- parentElement
- children
- nextSibling
- previousSibling
- style

DOM: querySelectorAll()

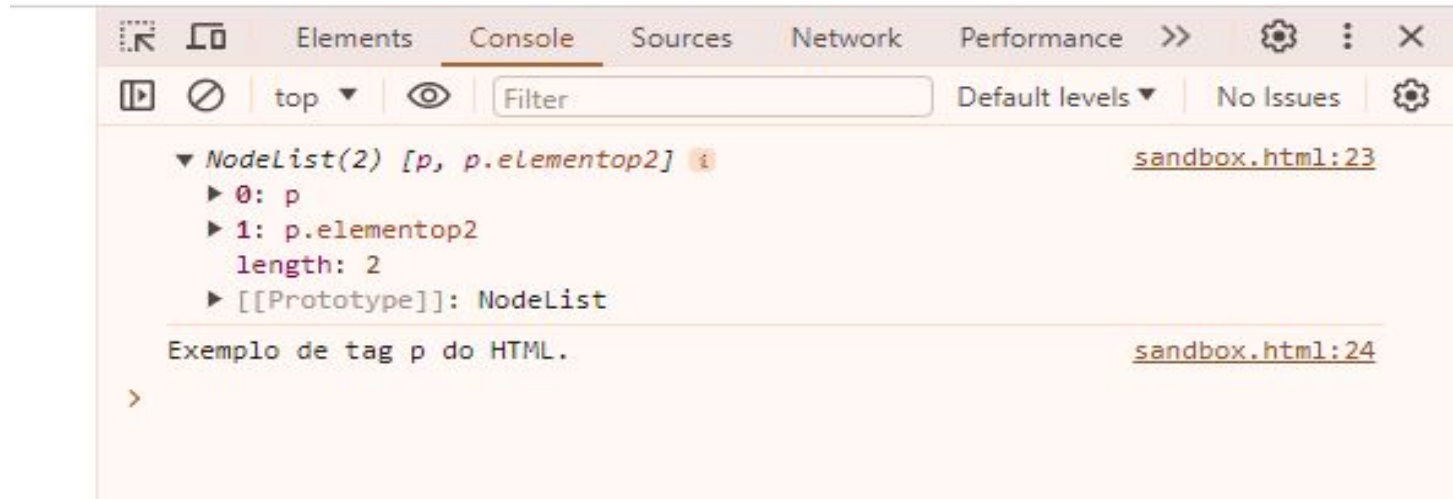
Permite selecionar **TODOS** que satisfaz a query.
Deve-se utilizar a sintaxe do CSS.

```
<h1>Exemplo de site</h1>
```

```
<p>Exemplo de tag p do HTML. </p>
```

```
<p class="elementop2">Exemplo de tag p do HTML 2. </p>
```

```
let todasTagP = document.querySelectorAll("p");  
console.log(todasTagP);  
console.log(todasTagP[0].innerText);
```



DOM: querySelectorAll()

Outro exemplo

```
<p>Exemplo de tag p do HTML. <a href="http://exemplo...">meu link</a> </p>
```

```
<p class="elementop2">Exemplo de tag p do HTML 2. </p>
```

```
todosLinksDentroDeP = document.querySelectorAll('p a')  
// todos os elementos 'a' que estão dentro de tags 'p'
```

```
console.log(todosLinksDentroDeP);  
console.log(todosLinksDentroDeP[0].href);
```

E mais um...

```
checkBox =  
document.querySelector('input[type="checkbox"]')  
//Seleciona o primeiro checkbox da pagina.
```

```
console.log(checkBox);  
console.log(checkBox.checked);
```

```
//Se usar o querySelectorAll, não esquecer de acessar usando arrays.  
checkboxs[0].checked.
```

JS: Outras formas de acessar os elementos

- `document.getAttribute()` e `document.setAttribute()`
 - É o mesmo que acessar `meuObjeto.nomeAtributo`.
 - Exemplo:
 - `document.querySelector('img').src = 'xxx'`
 - é o mesmo que `meuObjeto.setAttribute('src', 'xxx')`.

`<p class="tagsp">Exemplo de tag p do HTML.</p>`

`<p class="elementop2">Exemplo de tag p do HTML 2. </p>`

`// Alterando a classe de um objeto`

`tagP = document.querySelectorAll('.tagsp')`

`console.log(tagP);`

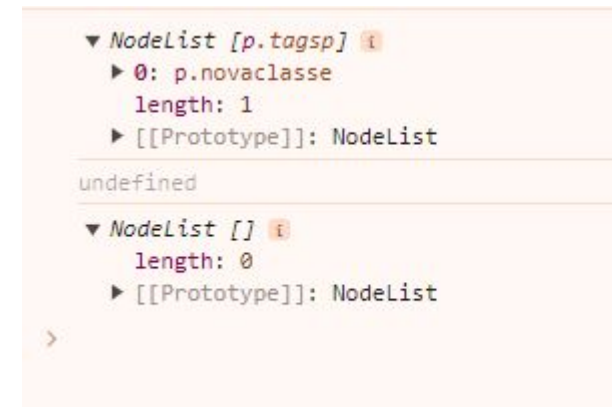
`console.log(tagP.class);`

`tagP[0].setAttribute('class', 'novaclasse');`

`novaTagP = document.querySelectorAll('.tagsp')`

`console.log(novaTagP);`

`//Antes tínhamos um objeto p com a classe tagsp, agora não temos mais.`



JS: Outros detalhes...

- Ao alterar o style de um elemento por JavaScript, é como se inseríssemos um *CSS in-line*, ou seja, individual para cada elemento.
 - Mas e se precisarmos redefinir atributos de vários elementos?
- Podemos incluir ou excluir classes de elementos usando o **classList**.
 - Adiciona ou remove classes de um objeto.
 - Ao adicionar uma classe, o elemento herda todas as características da classe.
 - Por exemplo, “.purple” define a cor purple para o objeto.
 - Para adicionar: **h2.classList.add('purple')**
 - Ou **h2.classList.toggle('purple')**.
 - O toggle adiciona ou remove, se já tiver ele remove, se não tiver, ele adiciona.

JS: Outros detalhes...

- innerHTML
 - Texto em formato HTML
- innerText vs textContent
 - **innerText**: sensível ao contexto, só mostra o que está sendo exibido no momento da query
 - **textContent**: devolve tudo, o que está sendo visto ou não.
- Por exemplo, se definir um texto com “***display: none***”, ele irá sumir do innerText, mas não do textContent.

Resumo:
funções, classes, construtores e
seleção de elementos.

EXERCÍCIO

- Usando 2 `<input>`, receber preço e quantidade e calcular o valor total ($\text{preço} * \text{quantidade}$) e exibir o valor total abaixo.
- Quando um deles for alterado, alterar automaticamente o valor total.

EXERCÍCIO

- Crie uma página web que mostra uma lista de alunos usando `<ul>` e `<li>`. Após criar pelo menos 6 nomes de alunos, crie o código CSS e Javascript para alterar a cor do texto para vermelhos dos alunos que possuem uma posição par na lista.

Dica: use o `querySelectorAll` e o `for`

Lista de Alunos

João

Maria

Pedro

Ana

Lucas